

SDR Platform — Applications Introduction

Contents

SDR Platform — Applications Introduction	1
1. The shared substrate	1
2. Application 1 — Reliable Digital Link	2
3. Application 2 — STLC Digital Over-the-Air Computation	4
4. How the two applications unify	6
5. Reference appendix	6

SDR Platform — Applications Introduction

A single software-defined-radio physical layer (`sdr_system`, C++/UHD) carries several very different **applications**. This document introduces each one in detail: what it computes, how it uses the radio, the exact signal chain it exercises, the hardware topology it runs on, and its current status. It is written for a collaborator coming to the code fresh (OSU experiments / UnionLabs deep-dive).

Two applications exist today, and they sit at **opposite ends of the design space**:

	Application 1 — Reliable Digital Link	Application 2 — Over-the-Air Computation
Paradigm	decode-and-forward (one packet, one receiver)	analog superposition (many senders, one function)
Transmitters active at once	1 (per slot)	N (simultaneously)
What the receiver produces	the delivered bytes + an ACK	an aggregate (the sum) of all senders
Receiver work	full sync + demod + FEC decode + CRC	CSI + a fixed linear combine
Hosts	MARL random access, OSU, Federated Learning	STLC digital AirComp (Lee–Lee–Jung, WCL 2019)
Status	validated over the air	design mapped; new blocks to build

The rest of the document is: the shared substrate both applications stand on (§1), then each application in full (§2, §3), then how they unify under one abstraction (§4), and a reference appendix (§5).

1. The shared substrate

Everything below the application logic is common. An application never re-implements DSP; it either drives the monolithic engine through a CLI wrapper, or composes the PHY as GNU-Radio-style blocks in Python.

1.1 The PHY engine (`sdr_system`, C++)

A complete two-radio digital link. Full detail lives in `../SYSTEM_REFERENCE.pdf`; the parts the applications rely on:

```
TX: message -> chunk -> CRC-16 -> [FEC encode] -> bits->symbols (modulator)
    |-- single-carrier: RRC pulse-shape -> preamble -> USRP TX
    |-- OFDM:          QAM -> IFFT+CP frame -> USRP TX
```

```

RX:  USRP RX -> energy gate -> AGC -> sync
      |-- single-carrier: matched filter -> timing -> [equalize] -> demod
      |-- OFDM:          Schmidl-Cox + CFO -> FFT -> 1-tap eq -> pilot CPE -> demod
      -> [FEC decode] -> CRC-16 -> (ARQ: ACK if OK) -> reassemble

```

- **Modulation:** BPSK, QPSK, 8-PSK, 16/32/64/128/256-QAM, and differential DBPSK / DQPSK / 8-DPSK / PI4-QPSK.
- **Waveform:** single-carrier (RRC, roll-off 0.25) or OFDM (per-subcarrier pilots that track a free-running carrier-frequency offset).
- **FEC:** rate-1/2 convolutional (K=7) + Viterbi, **LDPC** (IRA/staircase, normalized min-sum belief propagation), and **turbo** (punctured PCCC, iterative max-log-MAP BCJR). Hard or soft decision; tunable block size k, iterations, and normalization.
- **Reliability:** CRC-16, stop-and-wait ARQ, ACK carried over TCP or over RF.
- **Channel awareness:** energy sensing, freq_scan.py, live EVM and pre-/post-FEC BER.

Free-running-LO fact that shapes both applications: the two radios' local oscillators run independently. Coherent single-carrier QPSK then fails (~50 % BER) because the carrier phase spins; the two remedies are **differential** modulation (DQPSK/DBPSK — no absolute phase reference needed) or **OFDM** (pilots track the offset). This is why differential modes are first-class, and it directly informs the AirComp design in §3.

1.2 The block API (pyphy)

Every DSP stage is also exposed as a numpy-in / numpy-out function via pybind11, so an application can compose the chain in Python and insert its own operations between stages (exactly the GNU Radio model):

```

framing : frame / unframe
fec      : fec_encode / fec_decode / fec_decode_soft      (conv|ldpc|turbo)
mod      : modulate / demodulate / soft_llr
sc       : rrc_tx / rrc_rx
sync     : preamble / acq / cfo_correct / phase_correct
ofdm     : ofdm_mod / ofdm_demod
radio    : Radio('tx'|'rx', ...).transmit(wave) / .capture(n)      (WITH_UHD build)

```

The same C++ the binary uses, so behaviour is identical. This block API is the substrate Application 2 will be built on (custom transmit math + raw multi-antenna capture + custom receiver estimation are impossible in the monolithic binary but natural here).

1.3 The abstraction layer (in progress)

For the UnionLabs collaboration we are standardizing how an algorithm plugs into the PHY: an SdrApp implements three methods — `next_payload()` -> `float32[]` (PHY grabs it to transmit), `on_payload(float32[])` (received data handed back), `on_result(ack)` (delivered / collision, i.e. the reward) — and declares one `PayloadSpec(dtype, shape)`. Adding an application becomes "3 methods + 1 spec"; nothing below the seam changes. §4 revisits this once both applications are on the table, because Application 2 stretches the contract.

2. Application 1 — Reliable Digital Link

2.1 What it is

A classic **decode-and-forward** link: one device transmits a packet, one access point recovers the exact bytes, and an ACK closes the loop. The "application" living on top decides *what* to send and *what the ACK means*. Three algorithms share this link, in two archetypes:

- **Data-transfer** — the payload is the information (a float32 vector). Uses `next_payload()` + `on_payload()`. Example: **Federated Learning**.

- **Control / random-access** — the payload is fixed; the *outcome* is the signal. Uses `next_payload() + on_result(ack)`, where the **ACK boolean is the reinforcement-learning reward**. Examples: **MARL** random access and **OSU**.

The full PHY chain of §1.1 is exercised end-to-end: framing, FEC, modulation, sync, demodulation, decoding, CRC, ARQ.

2.2 MARL random access (control archetype)

Multi-agent reinforcement learning over a contended channel. Topology is **N transmitting agents -> 1 receiving access point**.

- **The reward is the ACK.** Each agent fires a single-shot burst (`--max-attempts 1`); if its frame decodes and the AP routes back an ACK, that is a success (reward), otherwise a timeout means collision/loss. The policy — not the PHY — owns retransmission.
- **Per-agent ACK routing (1 RX -> N TX).** You do not need N receivers. The agent id is carried in payload byte 0; the AP runs the C++ decoder with role `rx --marl-report`, which prints one `[BURST] id=... idx tot nbytes hex=...` line per CRC-OK burst. `ap_multi.py` parses those lines and routes an ACK to the agent whose frame decoded. A collision produces no line, hence no ACK, hence a natural timeout.
- **Slotted time (slot_sync.py).** So contention is well defined, a standalone TCP slot clock broadcasts `SLOT <k>` every `--slot-ms`; agents block on `wait_slot()` at the top of each decision. The AP arbitrates at each slot boundary: `>=2` intents -> COLLISION (no ACK to anyone, even if one happened to decode via capture); exactly 1 intent that decoded -> real ACK; 1 intent that did not decode -> real loss; 0 -> idle. ACKs are emitted only at arbitration, so collisions are deterministic rather than dependent on RF-overlap timing.
- **Warm radios.** Both source and sink stay warm (LOs run continuously) so the CFO is stable and trackable — this lets coherent QPSK win (85 % vs DQPSK 62 % delivery on the directional link) instead of being forced onto differential by cold-LO jumps.
- **Observation vs payload.** The agent's RL state [`AoI`, `queue`, `channel-busy`] (`float32`) is *local* and is **not** transported — only the fixed burst crosses the air. This is the clean separation the abstraction layer formalizes.

2.3 OSU (control archetype)

OSU is the same control loop as MARL with a different policy swapped in. It observes its state, decides transmit-or-defer, fires one burst through the adapter, and reads back delivered/collision as reward. It is the reference application we are porting onto the `SdrApp` contract first; the goal of the deep-dive is to make "bring your own policy" a 3-methods-and-a-spec exercise.

2.4 Federated Learning (data-transfer archetype)

FedAvg of an MNIST classifier over the radio (`fl.py` / `fl_core.py`, `numpy-only`, `torch-free`).

- **What crosses the air:** model updates as `float32` vectors. In the default **compressed** mode all nodes share an init seed and exchange only **top-k sparse deltas** with error feedback both directions; the server FedAvg-averages the sparse client deltas and broadcasts the sparse aggregate. This is ~ 20 KB/delta versus ~ 200 KB for the full model. Mock-validated live on MNIST: **0.71 -> 0.93 accuracy in 20 rounds, 2 clients** (beating full-model 0.925).
- **Per-direction channel.** Because the rig has an RX-only N210 and TX-only B210s, the only RF path is B210 -> N210 (uplink). `--uplink {wireless,tcp}` / `--downlink {wireless,tcp}` lets uplink go over RF while the model broadcast returns over plain TCP (the N210 never has to transmit). All-TCP mode runs the whole protocol radio-free and was validated end-to-end: MNIST 0.92 in 12 rounds.
- **Waveform choice.** `--waveform sc` uses differential DQPSK (default); `--waveform ofdm` uses coherent QPSK, whose per-subcarrier pilots track the free-running CFO — so OFDM-QPSK works where single-carrier coherent QPSK would fail.

2.5 Hardware topology and operating points

- **Radios:** N210 (addr=192.168.20.2, subdev A:0, gain 0–31.5 dB, rate 100e6/int) as the access point / server; B210s (serial=30CD424, 30CD3F7, subdev A:A, gain up to ~89 dB, flexible rate, 2×2 MIMO) as the agents / clients.
- **Rates:** the MARL stack uses a self-consistent 1.6e6 sample / 0.8e6 symbol default; the N210 needs its exact 2e6/1e6 pairing (1.6e6 does not snap on the 100 MHz clock).
- **Operating envelope:** validated per-scheme B210 gains and EVM ceilings at 915 MHz; dense constellations (16-QAM and up) need a cable, coherent QPSK needs the warm regime.

2.6 Status

MARL: per-agent ACK routing, slot sync and logical collision arbitration are built and sim-validated; hardware pieces (N210 decoded a DQPSK burst from a B210, [BURST] line emits and parses) are proven. FL: compression and the TCP/wireless channel split are done and validated over TCP; the wireless run is command-ready and waiting on radio time. This application is mature.

3. Application 2 — STLC Digital Over-the-Air Computation

Y.-S. Lee, K.-H. Lee, and B. C. Jung, "Space-time coded over-the-air computation with receive diversity for 6G massive IoT networks," *IEEE Wireless Communications Letters*, vol. 15, pp. 31–35, 2026. (Companion work: "Digital Over-the-Air Computation with Space-Time Processing for 6G Massive IoT Networks.")

3.1 What it is — and how it differs from Application 1

Over-the-air computation (AirComp) turns the wireless medium itself into a calculator: many IoT sensors transmit **simultaneously**, their signals **superimpose** in the air, and the access point reads a **function** of them — the **sum / average** — *without ever decoding an individual packet*. Where Application 1 asks "did this one device's bytes arrive?", Application 2 asks "what is the aggregate of all devices' measurements?" For massive IoT (thousands of sensors reporting to a fusion center) that is exactly the quantity of interest, and computing it over the air removes the per-device access bottleneck.

This specific scheme is **digital** AirComp (values are quantized, not sent as raw analog amplitudes — far more noise-robust than classical analog AirComp) combined with a **Space-Time Line Code (STLC)** for **receive diversity**.

3.2 The scheme

Each sensor (single transmit antenna, knows only its own channel):

1. **Quantize** its measurement to bits.
2. **Bit-slice** the bits into **low-order-modulated** symbols — the paper focuses on **BPSK and DBPSK**.
3. **STLC-encode** across **2 time slots**, precoding with its *own* channel estimate (local CSI at the transmitter).
4. **Transmit** — time-aligned with every other sensor, so the signals **superimpose** at the access point.

Access point (>= 2 receive antennas):

5. **Capture** the 2 time slots on the diversity antennas.
6. **Linear-combine** (the STLC combiner) — a *fixed*, *CSI-free* operation — to detect the **superimposed (summed)** symbols with full diversity.
7. **De-map / de-bit-slice / de-quantize** to reconstruct the **aggregate**.

3.3 Why STLC, and why the receiver is nearly free

A **Space-Time Line Code** is the transmit-side dual of the familiar space-time *block* code (Alamouti). In an STBC the transmitter has no channel knowledge and the *receiver* does the channel-aware combining. In an **STLC** the

transmitter uses **local** CSI to precode across the 2 time slots so that the receiver achieves full diversity with a **simple, CSI-free linear combine** — the channel effects cancel in the combining by construction.

Two consequences make this a good fit for massive IoT and for our hardware:

- **Only local CSI at the transmitter** (each sensor its own channel) — no global CSI, low overhead. This is Bang Chul Jung’s group’s signature technique.
- **The receiver does almost no work.** As the collaborator confirmed: *in their frame there is no data processing on the receive side except the CSI computation.* The AP estimates CSI (the sounding that feeds the transmit precoding), does the fixed linear combine, and de-quantizes. There is **no per-device equalization, no FEC, no ARQ** on this path — the diversity is manufactured at the transmitter.

DBPSK is significant for us: it is differential, so the AP needs **no absolute phase reference**. Given our free-running LOs (§1.1), this likely removes the shared-10 MHz-reference requirement that coherent superposition would otherwise impose — the differential detection tolerates the independent oscillators. Time alignment across sensors is still required, and the STLC precoding still needs the local CSI.

3.4 Mapping onto our platform

Because the receiver is so light and the modulation is low-order, most of Application 1’s heavy machinery (FEC, the full sync/CFO/phase/demod pipeline, ARQ/CRC) is **not on this application’s critical path**. What we already have, and what we must add:

Need	Status
Low-order modulation (BPSK / DBPSK)	have — <code>pyphy.modulate</code>
Custom transmit preprocessing (quantize, bit-slice, STLC)	have — compose on the <code>pyphy</code> blocks
Simultaneous, slot-aligned transmission from N sensors	have — reuse <code>slot_sync.py</code> + <code>Radio.transmit</code>
Raw superposition capture at the AP	partial — capture is single-antenna today
Receive diversity (≥ 2 RX antennas)	build — 2-channel capture (<code>capture2</code>) on B210/X310 (both 2×2)
Local CSI estimation (channel sounding)	expose — wrap existing pilot channel-estimation as a block
STLC encode / combine, sum reconstruction	build — small <code>numpy</code> blocks

So the realistic work is **one engine addition (a synchronized 2-channel RX capture) + three small numpy blocks (`stlc_encode`, `stlc_combine`, and the quantize/bit-slice/aggregate app logic) + wiring the sounding step**. None of it touches the `sdr_system` decode path; it all lives on the block API of §1.2.

3.5 Concrete build list

- **Engine:** `Radio.capture2(n) -> (ant0, ant1)` — a synchronized 2-channel UHD RX stream (the one real engine addition; receive diversity needs it).
- **Blocks (numpy):**
 - `stlc_encode(sym, h)` — device side, 2-slot precoding from local CSI.
 - `stlc_combine(y0, y1[, ...])` — AP side, the fixed CSI-free linear combiner -> superimposed symbol.
 - `estimate_channel(rx, pilot) -> h` — expose the existing preamble/pilot estimator.
 - `quantize / bit_slice / dequantize / aggregate` — pure-Python application logic.
- **App loop:** `slot_sync` aligns the sensors -> each `stlc_encode`s its quantized value and fires -> AP `capture2` -> `stlc_combine` -> de-quantize -> the aggregate. Validate a 2-sensor -> 1-AP sum in loopback the same way the SC/OFDM chains were validated (0 errors through a known offset).

3.6 Open questions (need the paper’s equations to finalize)

1. **How is local CSI obtained** — TDD reciprocity (sensor estimates from an AP downlink pilot) or AP estimates-then-feeds-back? This decides whether the AP transmits at all, which matters given our RX-only N210 vs the TX-capable B210s.
2. **Number of AP receive antennas M** (diversity order is $2M$) — two, or more?

3. The exact **STLC precoding matrix** and **combiner** (the 2-slot formulas), the **quantization + bit-slicing** map, and what **synchronization** the system model assumes.

3.7 Status

Paradigm analyzed and mapped to concrete platform work; the block API is the right substrate. Not yet implemented — blocked on the paper’s precise STLC/estimator equations and on building the 2-channel capture. This is the immediate next application to construct.

4. How the two applications unify

Application 1 fits the existing two archetypes cleanly:

- **Data-transfer** (FL): `next_payload()` + `on_payload()`.
- **Control / random-access** (MARL, OSU): `next_payload()` + `on_result(ack)`, `ACK = reward`.

Application 2 does **not** fit either — its “output” is neither delivered bytes nor an ACK, but a **computed aggregate**, produced by **many senders transmitting at once** into a **multi-antenna** receiver. It is a **third archetype: compute / aggregation**. This is a useful stress test for the UnionLabs abstraction: the layer must support *simultaneous multi-node transmission* and *multi-antenna reception*, not only point-to-point. Concretely, the SdrApp contract extends so that a sensor contributes an STLC-encoded quantized value and the AP “receiver” is a combine-and-aggregate step rather than decode-one-packet.

Stated as one seam:

Archetype	Senders	Receiver output	Example
Data-transfer	1	the bytes	Federated Learning
Control / random-access	1 of N (contending)	ACK bool (reward)	MARL, OSU
Compute / aggregation	N simultaneous	an aggregate	STLC AirComp

Getting all three behind one interface is the point of the abstraction layer.

5. Reference appendix

5.1 pyphy block reference

Group	Blocks
Framing	<code>frame</code> , <code>unframe</code>
FEC	<code>fec_encode</code> , <code>fec_decode</code> , <code>fec_decode_soft</code> , <code>fec_encoded_len</code> (conv ldpc turbo, k)
Modulation	<code>modulate</code> , <code>demodulate</code> , <code>soft_llr</code>
Single-carrier	<code>rrc_tx</code> , <code>rrc_rx</code>
Sync	<code>preamble</code> , <code>acq</code> , <code>cfo_correct</code> , <code>phase_correct</code>
OFDM	<code>ofdm_mod</code> , <code>ofdm_demod</code> , <code>ofdm_data_per_sym</code>
Radio (WITH_UHD)	<code>Radio(role, args, freq, rate, symbol_rate, gain, subdev, ant) . .transmit() . .capture()</code>
To add (App 2)	<code>capture2</code> , <code>stlc_encode</code> , <code>stlc_combine</code> , <code>estimate_channel</code>

Build: `phy/bindings/build.sh` (DSP only, builds anywhere) or `WITH_UHD=1 bindings/build.sh` (adds the Radio block, on the lab host). Worked flowgraph: `phy/python/phy_flow_example.py`.

5.2 Hardware inventory

Device	Role	Address / serial	Subdev	Rate	Notes
N210	AP / server (RX-only on the rig)	addr=192.168.20.2	A:0	2e6/1e6 exact	gain 0–31.5 dB; 100e6/int clock
B210 #1	agent / client (TX)	serial=30CD424	A:A	flexible	gain ~0–89 dB; 2×2 MIMO
B210 #2	agent / client (TX)	serial=30CD3F7	A:A	flexible	2×2 MIMO (a diversity-RX can
X310	(optional)	addr=	—	200e6/int	2×2 MIMO

5.3 Where to read the code (bottom -> top)

sdr_system (C++ engine, --help surface) -> sdr.py (auto-generated CLI wrapper) -> marl_phy.py / real_channel.py (WarmSource / AccessPoint adapters) -> marl_env.py (obs / action / reward around the ACK) -> proposed phy_link.py (the SdrApp + PayloadSpec + Codec contract). Full engine reference: ../SYSTEM_REFERENCE.pdf.